현대 강화학습 자율이동체 구현 워크숍

DDPG 기반 무인이동체 제어

Chaemoon Im
Korea University, School of Electrical Engineering
Artificial Intelligence and Mobility Laboratory
anscodla0314@korea.ac.kr

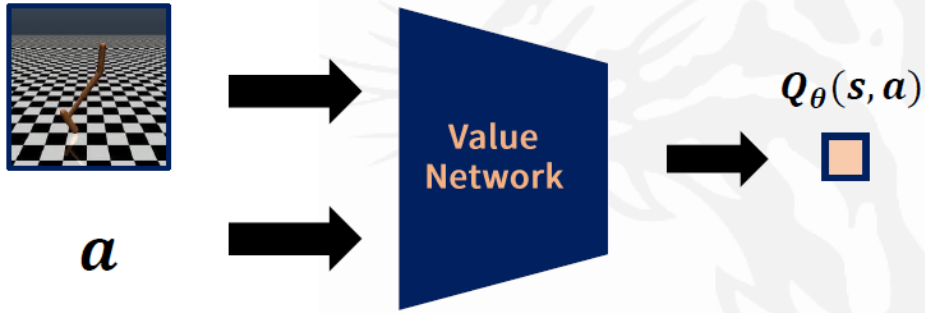
목차

Deep Deterministic Policy Gradient

Implementation

Q&A

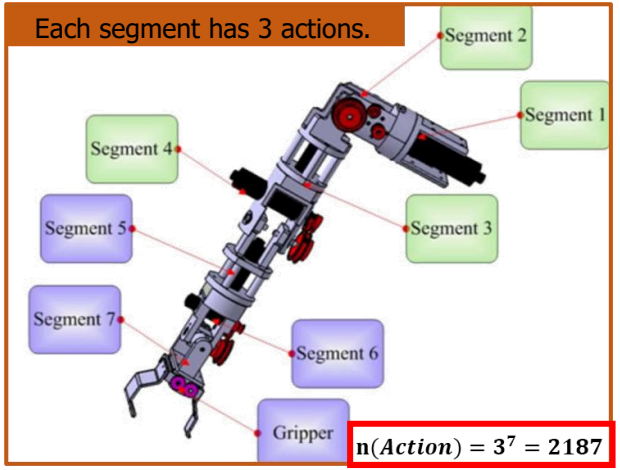
- **Limitations of Existing Algorithm (Deep Q-Network, DQN)**
  - **이산 공간밖에 다룰 수 없음**
    - DQN은 행동이 연속적인 경우에만 사용할 수 있음
  - **연속 공간을 표현하기 위한 이산화 과정에서 행동 차원 급격 증가**
    - 연속 공간을 이산화하여 DQN의 입력으로 사용할 수 있지만,
      다루고자 하는 공간이 커질수록 행동 차원이 급격히 증가함

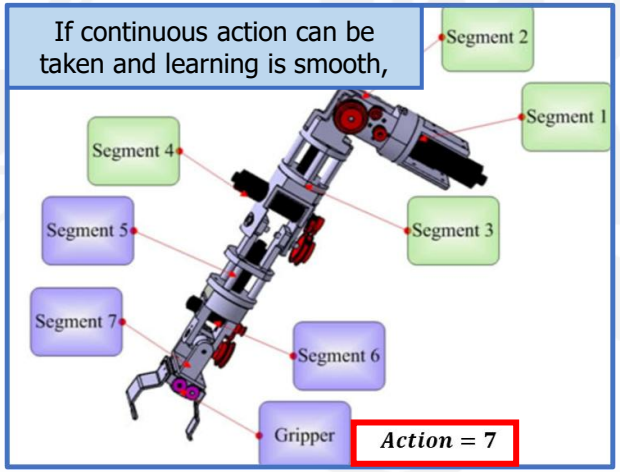


- **Solution : Deep Deterministic Policy Gradient (DDPG)**
  - Actor-Critic 모델을 사용
    - ✓ Actor: **현재 상황에서 무엇을 해야 할지 정책을 학습**하는 역할
    - ✓ Critic: **무엇을 해야 할 지 결정하는 데 도움을 주는** Q(s,a)를 학습하는 역할

→ DQN에서는 불가능했던 연속적인 정책 $\pi(a|s)$ 이 가능해짐.

→ 연속 공간에서도 보다 원활하게 Q(s,a) 함수의 학습이 가능함.





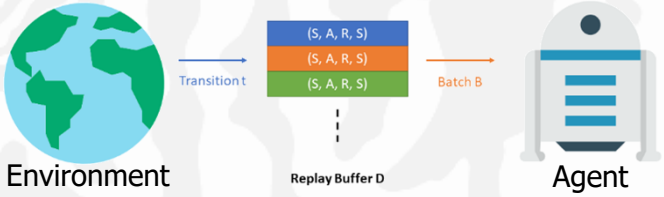
Deep Deterministic Policy Gradient (DDPG)

- Deterministic Policy
  - **Cost Function**, $J(\mu_\theta)$
    - ✓ Q(s,a) 가 아니라, 정책 μ를 직접적으로 사용하여 손실함수 정의

      → $J(\mu_\theta) = \mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$

  - **Policy Gradient**, $\nabla_\theta J(\mu_\theta)$
    - ✓ 해당 정책은 평균 보상을 증가시키는 방향으로 업데이트됨.

      → $\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim p^{\pi_\theta}}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q(s,a)|_{a=\mu_\theta(s)}\right]$



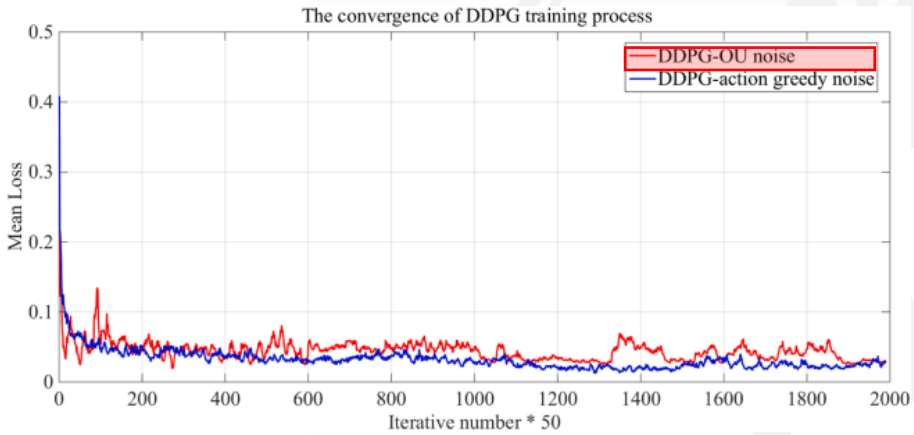
  - **Sample-Based Learning and Off-Policy Learning**
    - ✓ 특정 정책에 따라 모인 데이터는, 다른 정책을 학습할 때도 사용 가능 (Off-Policy).
    - ✓ DQN과 비슷하게 Replay Buffer를 도입, 모인 데이터를 통해 학습 진행

Deep Deterministic Policy Gradient (DDPG)

- **Noise process**
  - 이산적인 정책만으로 불가능한 탐색(exploration)을 위해, 행동에 노이즈 추가.

    → $\mu'(s_t) = \mu(s_t|\theta_t^{\mu}) + \mathcal{N}$.
  - 이 때, N은 통상적으로 OU noise (Ornstein-Uhlenbeck noise) 를 사용

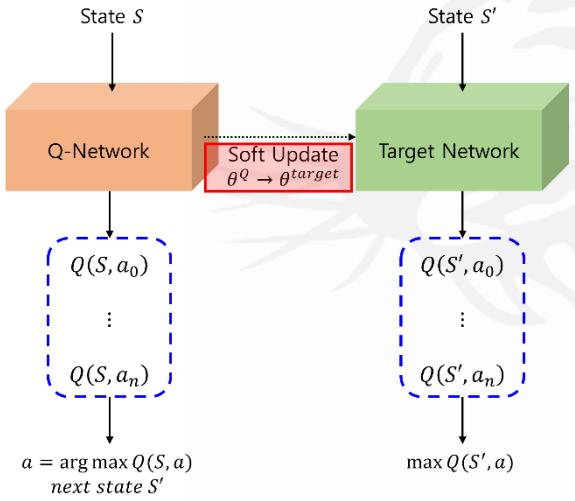
→에이전트가 특정 상태에 머무르지 않고, 다양한 시도를 통해 보다 나은 행동을 학습할 수 있도록 유도함.



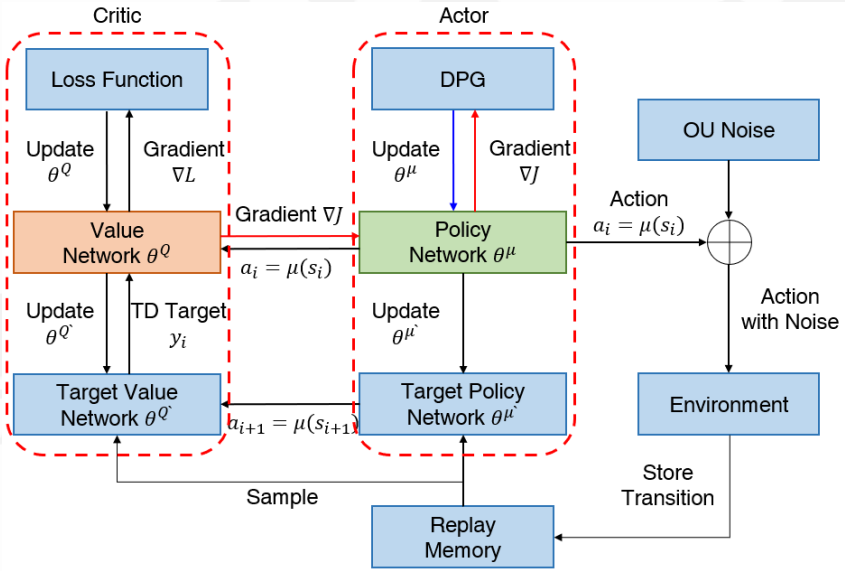
- **Soft target update**
  - DQN과 마찬가지로, Actor 및 Critic 네트워크를 학습하기 위한 Target Actor, Critic 네트워크 도입.
    - ✓ 같은 구조를 가지고, **더 느리게 파라미터를 업데이트함.**

$$\rightarrow \theta' \leftarrow \tau\theta + (1-\tau)\theta', with\ \tau \ll 1 \text{ (typically close to 0.001)}$$

→ Target network의 급격한 변화를 방지해 보다 안정적으로 학습이 이루어질 수 있도록 도움.

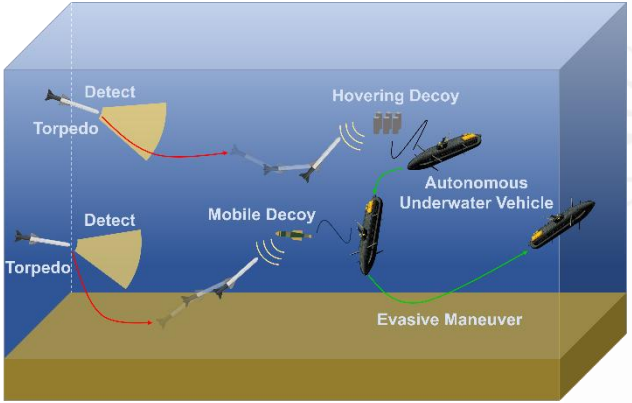


- **전반적인 학습 과정**
  1. Actor는 현재 정책에 따라 데이터 $(s_t, a_t, r_t, s_{t+1}, done)$ 수집
  2. 이 때, 탐색을 위해 Actor가 도출한 행동에 일정 노이즈를 더해서 환경과 상호작용함
  3. Replay Buffer에 충분한 데이터가 모이면, Actor 및 Critic 업데이트
     1. Actor는 자신의 행동이 Critic에게 **더 좋은 평가를 받을 수 있도록** 업데이트 진행
     2. Critic은 **보다 정확하게 Q(s,a)를 추정**하도록 업데이트 진행
  4. Soft Update를 통해 Target actor, critic 네트워크 업데이트

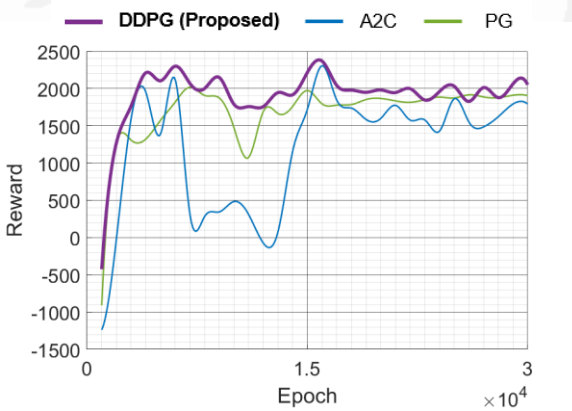


Deep Deterministic Policy Gradient (DDPG)

- **실제 논문 구현 환경**
  - 두 대의 어뢰를 잠수함이 피하는 환경을 가정
  - 잠수함은 수중 내에서 3차원으로 움직일 수 있으며, 기만기를 발사하여 어뢰 회피 가능
  - 어뢰는 잠수함 근처 일정 반경 내에서 생성되며,
    잠수함이 격침될 경우 에피소드 종료
  - 이와 **유사한 모의 환경 내**에서, DDPG 알고리즘 구현



실험 환경 모식도



실험 결과

➢ Box World

- Objective) 2D 공간 내에서 어뢰 회피
- State) 8차원 실수 벡터

$$S = \{\text{현재 잠수함 및 어뢰의 위치, 선체의 각도}\}$$

- Action) 1차원 실수 스칼라

$$A = \left[-\frac{\pi}{2} \le a \le \frac{\pi}{2}\right], \text{ 잠수함 선체의 향하고자 하는 각도.}$$

- Reward)

$$Reward = -(\min(d_0, d_1)) + c_0 R_{hit} + c_1 R_{evade}$$

  ✓ d_0, d_1: 잠수함과 어뢰 0, 1 간의 거리
  ✓ R_{hit}, R_{evade}= 충돌 시 -100, 끝까지 회피에 성공할 시 100
- Termination Condition)
  ✓ 잠수함이 격침되었을 경우
  ✓ 현재 에피소드 수가 50을 넘길 경우 (성공적으로 어뢰 회피)

Implementation (Replay Buffer)

```python
18 class ReplayBuffer():
19     def __init__(self):
20         self.buffer = collections.deque(maxlen=buffer_limit)
21
22     def put(self, transition):
23         self.buffer.append(transition)
24
25     def sample(self, n):
26         mini_batch = random.sample(self.buffer, n)
27         s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []
28
29         for transition in mini_batch:
30             s, a, r, s_prime, done = transition
31             s_lst.append(s)
32             a_lst.append([a])
33             r_lst.append([r])
34             s_prime_lst.append(s_prime)
35             done_mask = 0.0 if done else 1.0
36             done_mask_lst.append([done_mask])
37
38         return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst, dtype=torch.float), \
39                 torch.tensor(r_lst, dtype=torch.float), torch.tensor(s_prime_lst, dtype=torch.float), \
40                 torch.tensor(done_mask_lst, dtype=torch.float)
41
42     def size(self):
43         return len(self.buffer)
```

Replay Buffer

- python.collections 라이브러리의 deque() class.
- DQN과 동일한 형태로 동작.

[Line 19-20] 초기화 수행.

[Line 22-23] $(s_t, a_t, r_t, s_{t+1}, done)$ 의 기록을 저장하는 put() 함수 정의.

Implementation (Replay Buffer)

```python
18 class ReplayBuffer():
19     def __init__(self):
20         self.buffer = collections.deque(maxlen=buffer_limit)
21 
22     def put(self, transition):
23         self.buffer.append(transition)
24 
25     def sample(self, n):
26         mini_batch = random.sample(self.buffer, n)
27         s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []
28 
29         for transition in mini_batch:
30             s, a, r, s_prime, done = transition
31             s_lst.append(s)
32             a_lst.append([a])
33             r_lst.append([r])
34             s_prime_lst.append(s_prime)
35             done_mask = 0.0 if done else 1.0
36             done_mask_lst.append([done_mask])
37 
38         return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst, dtype=torch.float), \
39                 torch.tensor(r_lst, dtype=torch.float), torch.tensor(s_prime_lst, dtype=torch.float), \
40                 torch.tensor(done_mask_lst, dtype=torch.float)
41 
42     def size(self):
43         return len(self.buffer)
```

Replay Buffer

[Line 25] 버퍼에서 데이터를 랜덤 샘플하는 sample() 함수 정의.
[Line 26] random.sample() 함수로 샘플링 진행.
[Line 27] 튜플 <s, a, r, s', done_mask>을 각각의 리스트로 쪼갤 수 있도록 사전 할당.

Implementation (Replay Buffer)

```python
18  class ReplayBuffer():
19      def __init__(self):
20          self.buffer = collections.deque(maxlen=buffer_limit)
21
22      def put(self, transition):
23          self.buffer.append(transition)
24
25      def sample(self, n):
26          mini_batch = random.sample(self.buffer, n)
27          s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []
28
29          for transition in mini_batch:
30              s, a, r, s_prime, done = transition
31              s_lst.append(s)
32              a_lst.append([a])
33              r_lst.append([r])
34              s_prime_lst.append(s_prime)
35              done_mask = 0.0 if done else 1.0
36              done_mask_lst.append([done_mask])
37
38          return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst, dtype=torch.float), \
39                  torch.tensor(r_lst, dtype=torch.float), torch.tensor(s_prime_lst, dtype=torch.float), \
40                  torch.tensor(done_mask_lst, dtype=torch.float)
41
42      def size(self):
43          return len(self.buffer)
```

Replay Buffer

[Line 29-30] mini_batch안에 있는 각 transition에 대해, 각각의 요소들을 분할.
[Line 31-36] 각 요소들을 분할하여 리스트에 별개로 저장함.
* a, r, done_mask만 []를 통해 리스트화해주는 것은 다른 요소들과 차원을 맞추기 위해서임.

Implementation (Replay Buffer)

```python
18  class ReplayBuffer():
19      def __init__(self):
20          self.buffer = collections.deque(maxlen=buffer_limit)
21
22      def put(self, transition):
23          self.buffer.append(transition)
24
25      def sample(self, n):
26          mini_batch = random.sample(self.buffer, n)
27          s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []
28
29          for transition in mini_batch:
30              s, a, r, s_prime, done = transition
31              s_lst.append(s)
32              a_lst.append([a])
33              r_lst.append([r])
34              s_prime_lst.append(s_prime)
35              done_mask = 0.0 if done else 1.0
36              done_mask_lst.append([done_mask])
37
38          return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst, dtype=torch.float), \
39                  torch.tensor(r_lst, dtype=torch.float), torch.tensor(s_prime_lst, dtype=torch.float), \
40                  torch.tensor(done_mask_lst, dtype=torch.float)
41
42      def size(self):
43          return len(self.buffer)
```

Replay Buffer

[Line 38-40] 각각의 요소를 담은 리스트들을 **torch.Tensor** 형태로 반환함.

Implementation (Actor)

```python
class Actor(nn.Module):
    def __init__(self, s_dim, a_dim):
        super(Actor, self).__init__()
        self.fc1 = nn.Linear(s_dim, 128)
        self.fc2 = nn.Linear(128, 64)
        self.mu = nn.Linear(64, a_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        mu = np.pi * torch.tanh(self.mu(x))
        return mu
```

Actor

- 외부 정보를 통해 행동 수행.
- 현재 상태 s_t를 기반으로 좋은 보상을 얻을 수 있는 정책 학습.

Implementation (Actor)

```python
45  class Actor(nn.Module):
46      def __init__(self, s_dim, a_dim):
47          super(Actor, self).__init__()
48          self.fc1 = nn.Linear(s_dim, 128)
49          self.fc2 = nn.Linear(128, 64)
50          self.mu = nn.Linear(64, a_dim)
51
52      def forward(self, x):
53          x = F.relu(self.fc1(x))
54          x = F.relu(self.fc2(x))
55          mu = np.pi * torch.tanh(self.mu(x))
56          return mu
```

Actor

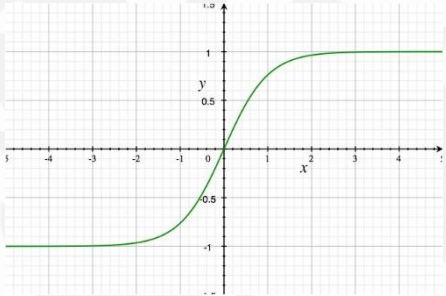
[Line 46-50] Actor 클래스 초기화.
[Line 47] **pytorch.nn.Module 클래스를 통해 해당 클래스 초기화 (pytorch의 자동 미분 기능 사용)**.
[Line 48-50] 각 layer의 사이즈 조정.
Input layer size : 8 (환경 정보)
Output layer size : 1 (행동 차원)
Two **hidden layer size** : 128, 64.

Implementation (Actor)

```python
45  class Actor(nn.Module):
46      def __init__(self, s_dim, a_dim):
47          super(Actor, self).__init__()
48          self.fc1 = nn.Linear(s_dim, 128)
49          self.fc2 = nn.Linear(128, 64)
50          self.mu = nn.Linear(64, a_dim)
51
52      def forward(self, x):
53          x = F.relu(self.fc1(x))
54          x = F.relu(self.fc2(x))
55          mu = np.pi * torch.tanh(self.mu(x))
56          return mu
```

Actor

[Line 52-58] forward() 함수 정의. 신경망의 출력을 반환하는 함수.
[Line 53-54] 신경망 계산을 통해 행동값 $\mu(s_t)$ 계산.
[Line 55] tanh() 함수를 통해 행동의 범위를 $(-\pi, \pi)$로 규격화함.



Tanh() function suppresses the action x into range [-1, 1]

Implementation (Critic)

```python
class Critic(nn.Module):
    def __init__(self, s_dim, a_dim):
        super(Critic, self).__init__()
        self.fc_s = nn.Linear(s_dim, 64)
        self.fc_a = nn.Linear(a_dim, 64)
        self.fc_q = nn.Linear(128, 32)
        self.fc_out = nn.Linear(32, 1)

    def forward(self, s, a):
        h1 = F.relu(self.fc_s(s))
        h2 = F.relu(self.fc_a(a))
        cat = torch.cat([h1, h2], dim=1)
        q = F.relu(self.fc_q(cat))
        q = self.fc_out(q)
        return q
```

Critic

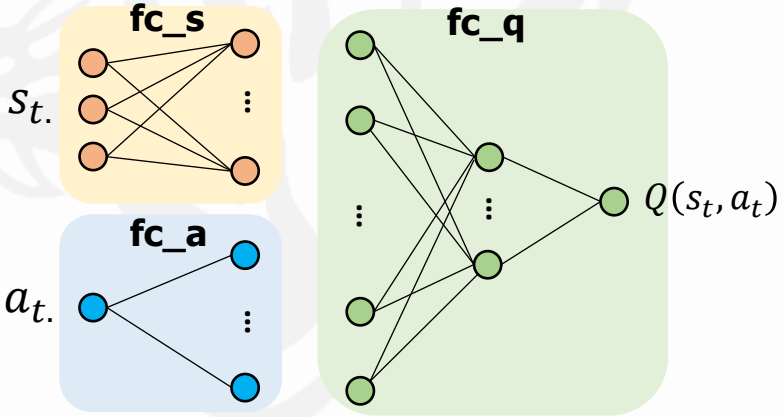
- 현재 State와 Action을 보고, Value function Q(s,a)를 예측하도록 학습을 진행함.
- Actor가 최적 정책을 찾는 데 도움을 주는 클래스.

Implementation (Critic)

```python
59  class Critic(nn.Module):
60      def __init__(self, s_dim, a_dim):
61          super(Critic, self).__init__()
62          self.fc_s = nn.Linear(s_dim, 64)
63          self.fc_a = nn.Linear(a_dim, 64)
64          self.fc_q = nn.Linear(128, 32)
65          self.fc_out = nn.Linear(32, 1)
66
67      def forward(self, s, a):
68          h1 = F.relu(self.fc_s(s))
69          h2 = F.relu(self.fc_a(a))
70          cat = torch.cat([h1, h2], dim=1)
71          q = F.relu(self.fc_q(cat))
72          q = self.fc_out(q)
73          return q
```

Critic

[Line 60-65] Critic 함수 초기화.
[Line 61] **pytorch.nn.Module 클래스를 통해 해당 클래스 초기화 (pytorch의 자동 미분 기능 사용).**
[Line 62] $s_{t.}$를 처리하는 신경망 fc_s 정의.
[Line 63] $a_{t.}$를 처리하는 신경망 fc_a 정의.
[Line 64-65] Line 62, Line 63 을 토대로 $Q(s,a)$를 추정하는 신경망 fc_q, fc_out 정의.

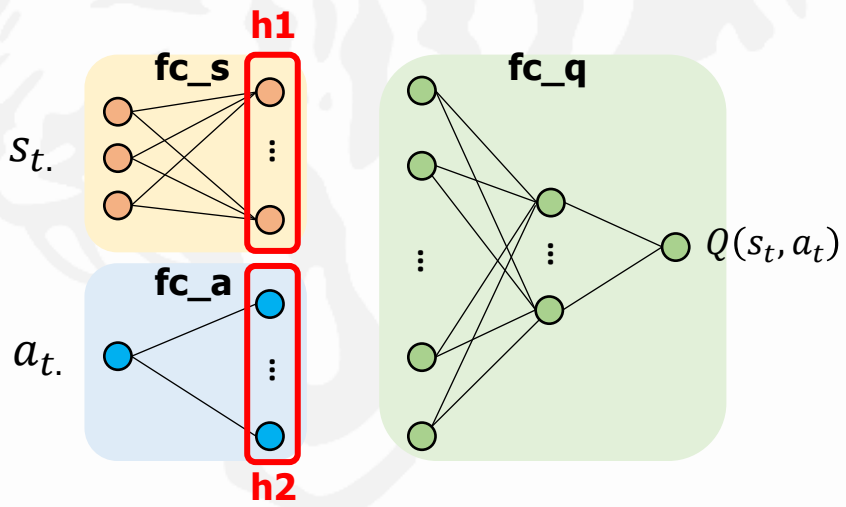


Implementation (Critic)

```python
59  class Critic(nn.Module):
60      def __init__(self, s_dim, a_dim):
61          super(Critic, self).__init__()
62          self.fc_s = nn.Linear(s_dim, 64)
63          self.fc_a = nn.Linear(a_dim, 64)
64          self.fc_q = nn.Linear(128, 32)
65          self.fc_out = nn.Linear(32, 1)
66
67      def forward(self, s, a):
68          h1 = F.relu(self.fc_s(s))
69          h2 = F.relu(self.fc_a(a))
70          cat = torch.cat([h1, h2], dim=1)
71          q = F.relu(self.fc_q(cat))
72          q = self.fc_out(q)
73          return q
```

Critic

[Line 67-73] **forward**() 함수 정의.
[Line 68] fc_s 신경망을 통해 **h1** 계산.
[Line 69] fc_a 신경망을 통해 **h2** 계산.

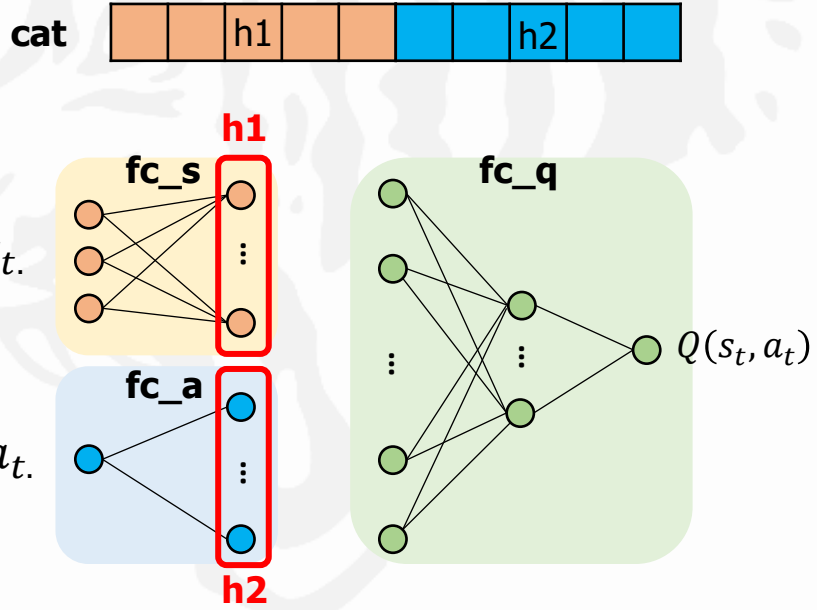


Implementation (Critic)

```python
59  class Critic(nn.Module):
60      def __init__(self, s_dim, a_dim):
61          super(Critic, self).__init__()
62          self.fc_s = nn.Linear(s_dim, 64)
63          self.fc_a = nn.Linear(a_dim, 64)
64          self.fc_q = nn.Linear(128, 32)
65          self.fc_out = nn.Linear(32, 1)
66
67      def forward(self, s, a):
68          h1 = F.relu(self.fc_s(s))
69          h2 = F.relu(self.fc_a(a))
70          cat = torch.cat([h1, h2], dim=1)
71          q = F.relu(self.fc_q(cat))
72          q = self.fc_out(q)
73          return q
```

Critic

[Line 70] h1, h2를 torch.cat 함수를 통해 결합.

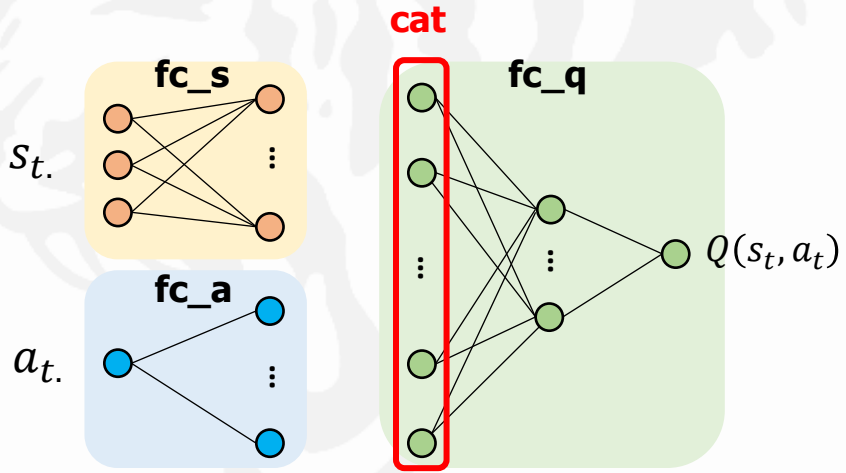


Implementation (Critic)

```python
59  class Critic(nn.Module):
60      def __init__(self, s_dim, a_dim):
61          super(Critic, self).__init__()
62          self.fc_s = nn.Linear(s_dim, 64)
63          self.fc_a = nn.Linear(a_dim, 64)
64          self.fc_q = nn.Linear(128, 32)
65          self.fc_out = nn.Linear(32, 1)
66
67      def forward(self, s, a):
68          h1 = F.relu(self.fc_s(s))
69          h2 = F.relu(self.fc_a(a))
70          cat = torch.cat([h1, h2], dim=1)
71          q = F.relu(self.fc_q(cat))
72          q = self.fc_out(q)
73          return q
```

Critic

[Line 71-72] 결합한 cat 값을 토대로, fc_q 및 fc_out 신경망을 통해 $Q(s_t, a_t)$ 계산.
[Line 73] 해당 값 반환.



Implementation (OU Noise)

OU Noise (Ornstein Uhlenbeck Noise)

- DDPG에서의 탐색을 위해 도입.
- **Mean-reversion process 임**
  (최초의 평균을 벗어날 경우
  다시 평균으로 회귀하려는 속성 가짐)

OU Noise

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

θ : speed of reversion to long-term mean
μ : long-term mean
σ : volatility

```python
class OrnsteinUhlenbeckNoise:
    def __init__(self, actor):
        self.theta, self.dt, self.sigma = 0.1, 0.01, 0.1
        self.actor = actor
        self.x_prev = np.zeros_like(self.actor)

    def __call__(self):
        x = self.x_prev + self.theta * (self.actor - self.x_prev) * self.dt + \
                self.sigma * np.sqrt(self.dt) * np.random.normal(size=self.actor.shape)
        self.x_prev = x
        return x
```

Implementation (OU Noise)

OU Noise (Ornstein Uhlenbeck Noise)

[Line 77] 파라미터 θ, σ, dt, 설정
θ : 얼마나 빠르게 평균으로 회귀할지 결정. 클수록 느림.
σ : volatility, 얼마나 flucatute 할지 결정.
dt : small value, 이산시간의 크기 결정.
[Line 78] **actor** 변수 도입. 행동 차원의 크기를 나타냄.
[Line 79] 최초 평균값인 **x_prev** 도입.

```
75  class OrnsteinUhlenbeckNoise:
76      def __init__(self, actor):
77          self.theta, self.dt, self.sigma = 0.1, 0.01, 0.1
78          self.actor = actor
79          self.x_prev = np.zeros_like(self.actor)
80
81      def __call__(self):
82          x = self.x_prev + self.theta * (self.actor - self.x_prev) * self.dt + \
83                  self.sigma * np.sqrt(self.dt) * np.random.normal(size=self.actor.shape)
84          self.x_prev = x
85          return x
```

OU Noise

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

Implementation (OU Noise)

OU Noise (Ornstein Uhlenbeck Noise)

[Line 81] __call__() 함수 정의. 이를 통해 클래스를 함수처럼 취급할 수 있게 됨 (ex) Ounoise())
[Line 82-84] OU noise 계산.
[Line 85] 해당 값 반환.

OU Noise

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

$$X_t = X_{t-1} + dX_t = X_{t-1} + \theta(\mu - X_t)dt + \sigma dW_t$$

```
75  class OrnsteinUhlenbeckNoise:
76      def __init__(self, actor):
77          self.theta, self.dt, self.sigma = 0.1, 0.01, 0.1
78          self.actor = actor
79          self.x_prev = np.zeros_like(self.actor)
80
81      def __call__(self):
82          x = self.x_prev + self.theta * (self.actor - self.x_prev) * self.dt + \
83                  self.sigma * np.sqrt(self.dt) * np.random.normal(size=self.actor.shape)
84          self.x_prev = x
85          return x
```

Implementation (train)

Train

- Actor 와 Critic 네트워크를 학습함.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1})) - Q(s_t, a_t)$$

```python
def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
    s,a,r,s_prime,done_mask  = memory.sample(batch_size)

    target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
    critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
    critic_optimizer.zero_grad()
    critic_loss.backward()
    critic_optimizer.step()

    actor_loss = -critic(s,actor(s)).mean()
    actor_optimizer.zero_grad()
    actor_loss.backward()
    actor_optimizer.step()
```

Implementation (train)

Train

[Line 88] Replay Buffer로부터 batch_size만큼 추출.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1})) - Q(s_t, a_t)$$

```python
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask  = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (train)

Train

[Line 90] Critic 네트워크를 위한 **target** 값 계산.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1})) - Q(s_t, a_t)$$

```
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask  = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (train)

Train

[Line 91] smooth_l1_loss를 통해 **critic_loss** 계산.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1})) - Q(s_t, a_t)$$

```
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask  = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (train)

Train

[Line 92] 옵티마이저 초기화.
[Line 93] 자동 미분 기능을 통해 **critic_loss 함수의 그래디언트 값 계산.**
[Line 94] 해당 그래디언트 값으로 gradient descent 수행.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1})) - Q(s_t, a_t)$$

```python
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (train)

Train

[Line 96] Actor의 loss를 계산.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1}) - Q(s_t, a_t)$$

```
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask  = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (train)

Train

[Line 97] 옵티마이저 초기화.
[Line 98] 자동 미분 기능을 통해 **actor_loss 함수의 그래디언트 값 계산.**
[Line 99] 해당 그래디언트 값으로 gradient descent 수행.

Actor network Loss

$$J(\mu_\theta) = -\mathbb{E}_{s\sim p^{\pi_\theta}}[r(s,\mu_\theta(s))]$$

Critic network Loss

$$r(s_t, a_t) + \gamma Q_{target}(s_{t+1}, \mu(s_{t+1}) - Q(s_t, a_t)$$

```
87  def train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer):
88      s,a,r,s_prime,done_mask  = memory.sample(batch_size)
89
90      target = r + gamma * critic_target(s_prime, actor_target(s_prime)) * done_mask
91      critic_loss = F.smooth_l1_loss(critic(s,a), target.detach())
92      critic_optimizer.zero_grad()
93      critic_loss.backward()
94      critic_optimizer.step()
95
96      actor_loss = -critic(s,actor(s)).mean()
97      actor_optimizer.zero_grad()
98      actor_loss.backward()
99      actor_optimizer.step()
```

Implementation (Soft Update)

[Line 101] **soft_update**() 함수 정의. 이는 Target netwrok의 파라미터를 업데이트하는데 사용됨.
[Line 102-103] 아래의 수식에 따라, 타겟 네트워크들의 파라미터를 업데이트함.

$$\theta_{target} = (1-\tau)\theta_{target} + \tau\theta$$

```
101 def soft_update(net, net_target):
102     for param_target, param in zip(net_target.parameters(), net.parameters()):
103         param_target.data.copy_(param_target.data * (1.0 - tau) + param.data * tau)
```

Implementation (Driving Code)

```
105 env = gym.make('Pendulum-v1', max_episode_steps=200, autoreset=True)
106 memory = ReplayBuffer()
107
108 critic, critic_target = Critic(), Critic()
109 critic_target.load_state_dict(critic.state_dict())
110 actor, actor_target = Actor(), Actor()
111 actor_target.load_state_dict(actor.state_dict())
112
113 score = 0.0
114 print_interval = 20
115 actor_optimizer = optim.Adam(actor.parameters(), lr=lr_actor)
116 critic_optimizer  = optim.Adam(critic.parameters(), lr=lr_critic)
117 ou_noise = OrnsteinUhlenbeckNoise(actor=np.zeros(1))
```

[Line 105] 환경 구성 (실제 예제에서는 다른 환경 사용).
[Line 106] Replay Buffer 정의.

[Line 108-109] Critic과 Target Critic 정의. 이후 Target Critic network의 파라미터를 Critic과 동일하게 설정.
[Line 110-111] Actor와 Target Actor 정의. 이후 Target Actor network의 파라미터를 Actor와 동일하게 설정.

[Line 113] 현재 보상 변수 **score** 정의.
[Line 114] 결과 출력 기간인 **print_interval** 정의.

[Line 115-116] Actor 및 Critic 개별의 **optimizer** 설정.
[Line 117] OU Noise 삽입을 위한 **ou_noise** 클래스 정의.

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 119-141] 총 10000번 반복. (임의로 조정할 수 있음)

[Line 120] 환경 초기화 및 초기 관측값 **s를 얻음**.
[Line 121] 언제 한 에피소드를 종료할지 결정하는 **done** 값 설정.
[Line 123] 한 에피소드 안에 몇 step이 실행되었는지 판단하는 **count 변수** 선언.

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 124] 한 에피소드 내 200 step 이내인 선에서, 아래의 while문 반복.
[Line 125] Actor가 현재의 정보 s_t **s** 를 바탕으로 행동 **a**를 결정.
[Line 126] **a 에 탐색을 위해 OU noise 삽입.**

$$\mu'(s_t) = \mu(s_t|\theta_t^{\mu}) + \mathcal{N}$$

a.item() ou_noise()[0]

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 127] a를 실행하고, 다음을 환경으로부터 얻음:
s_prime, s_{t+1}, 다음 상태
r, r_t, 보상
done, 이번 에피소드가 **종료**되었는지
Truncated, 이번 에피소드가 **중단**되었는지
info : 추가적으로 필요한 정보들
[Line 128] 해당 변수를 튜플 형태로 가공하여 $< s_t, a_t, \frac{r_t}{100}, s_{t+1}, is_done >$ Replay Buffer에 저장.

보상을 100으로 나누는 이유는, **너무 큰 값으로 인해 학습이 잘 되지 않는 경우**를 방지하기 위함.

Done : 최대 스텝 수를 채우거나 목적을 달성함 (어뢰로부터 회피)
Truncated : 목적을 달성하지 못하고 도중에 멈춤 (어뢰에 격침됨)

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 129] **score** 에 보상 **r** 더함.
[Line 130] **s** 에 **s_prime** 할당, 다음 step을 준비함.
[Line 131] 현재 스텝 수 **count**를 1 증가.

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic, critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 133] 만일 Replay Buffer에 충분한 양의 데이터가 모였다면,
[Line 134] 10번 training을 진행.
[Line 135] Actor, Critic 업데이트.
[Line 136-137] Target Actor, Critic을 Soft update 진행.

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 139-140] 일정 주기로 현재 에피소드 수, 현재 평균 점수 출력.
[Line 141] 다음 iteration에서의 평가를 위해 **score 값 0**으로 변경.

Implementation (Driving Code)

```python
for n_epi in range(10000):
    s, _ = env.reset()
    done = False

    count = 0
    while count < 200 and not done:
        a = actor(torch.from_numpy(s).float(
        a = a.item() + ou_noise()[0]
        s_prime, r, done, truncated, info = 
        memory.put((s,a,r/100.0,s_prime,done
        score +=r
        s = s_prime
        count += 1

    if memory.size()>2000:
        for i in range(10):
            train(actor, actor_target, criti
            soft_update(actor, actor_target)
            soft_update(critic,  critic_targ

    if n_epi%print_interval==0 and n_epi!=0:
        print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
        score = 0.0

env.close()
```

Print result is same as follows.

```
# of episode :20, avg score : -1478.3
# of episode :40, avg score : -1286.5
# of episode :60, avg score : -1395.8
# of episode :80, avg score : -1480.7
# of episode :100, avg score : -1493.0
# of episode :120, avg score : -1442.8
# of episode :140, avg score : -1363.1
# of episode :160, avg score : -1359.5
# of episode :180, avg score : -1199.2
```

Number of episode

Average Score

Implementation (Driving Code)

```
119 for n_epi in range(10000):
120     s, _ = env.reset()
121     done = False
122
123     count = 0
124     while count < 200 and not done:
125         a = actor(torch.from_numpy(s).float())
126         a = a.item() + ou_noise()[0]
127         s_prime, r, done, truncated, info = env.step([a])
128         memory.put((s,a,r/100.0,s_prime,done))
129         score +=r
130         s = s_prime
131         count += 1
132
133     if memory.size()>2000:
134         for i in range(10):
135             train(actor, actor_target, critic, critic_target, memory, critic_optimizer, actor_optimizer)
136             soft_update(actor, actor_target)
137             soft_update(critic,  critic_target)
138
139     if n_epi%print_interval==0 and n_epi!=0:
140         print("# of episode :{}, avg score : {:.1f}".format(n_epi, score/print_interval))
141         score = 0.0
142
143 env.close()
```

[Line 143] 실험 완료 시 환경 종료.

Thank you for your attention!

- More questions?
  - joongheon@korea.ac.kr
  - anscodla0314@korea.ac.kr
- More details?
  - https://joongheon.github.io/